

Financial Tweet Sentiment Classification

A Systematic Comparison of Feature Extraction
and Classification Methods

Group 42

TBD, Student ID

TBD, Student ID

TBD, Student ID

TBD, Student ID

Text Mining – 2025/2026

Contents

1	Introduction	2
2	Data Exploration	2
3	Data Preprocessing	3
3.1	Preprocessing Functions	3
3.2	Named Pipelines	4
3.3	Pipeline Comparison	4
4	Feature Engineering	5
4.1	Bag-of-Words Features	5
4.2	Word Embeddings	5
4.3	Transformer Encoders	5
5	Classification Models	6
6	Results and Analysis	7
6.1	Full Results Matrix	7
6.2	Key Findings	7
6.3	Per-Class Analysis	7
6.4	Embedding Geometry	8
6.5	Error Analysis	9
6.6	Learning Curve and Statistical Significance	9
7	Extra Work	10
7.1	Domain-Specific Transformer Encoders	10
7.2	Zero-Shot Pretrained Classification	10
7.3	Agentic Workflow	10
7.4	Decoder Model (Attempted)	11
8	Final Pipeline and Predictions	11
9	Conclusion	12

1. Introduction

Financial sentiment analysis on social media has practical applications in algorithmic trading, market surveillance and investor-sentiment tracking. Unlike general-purpose sentiment analysis, financial tweets present unique challenges: domain-specific vocabulary (cashtags, financial jargon), extreme brevity and an inherent class imbalance in which neutral commentary dominates.

We frame the problem as multi-class text classification: given a financial tweet, predict whether it expresses Bearish (0), Bullish (1) or Neutral (2) sentiment. Our approach evaluated 32 combinations of feature-extraction methods and classifiers using 5-fold stratified cross-validation, with macro-F1 as the primary metric.

The best pipeline — Twitter-RoBERTa embeddings combined with XGBoost — achieves a macro-F1 of **0.794**, outperforming traditional approaches by a substantial margin and demonstrating that domain-aligned pretrained representations matter more than classifier sophistication.

2. Data Exploration

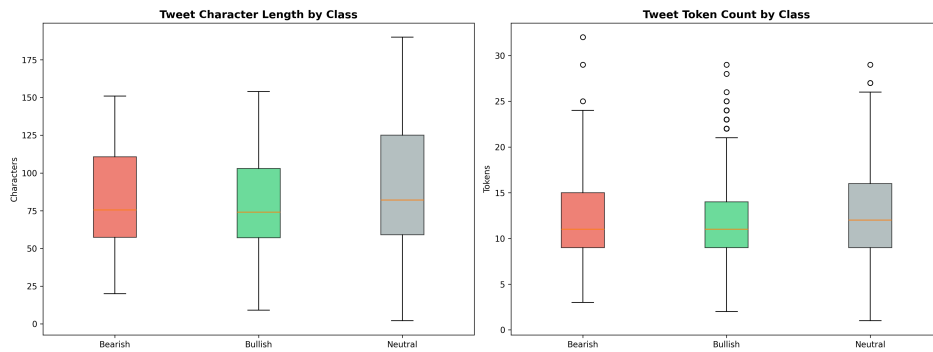
The dataset comprises 9,543 labeled training tweets and 2,388 unlabeled test tweets across three sentiment classes. It exhibits a strong class imbalance: Neutral tweets (64.7%) outnumber Bearish tweets (15.1%) by a factor of $4.3\times$ (Table 1). Two consequences shape the rest of the pipeline: (i) we use `class_weight='balanced'` in every classifier, and (ii) we adopt macro-F1 rather than accuracy as the primary metric, since a naive all-Neutral classifier would already reach 64.7% accuracy while failing entirely on the minority classes.

Class	Count	Proportion
Bearish (0)	1,442	15.1%
Bullish (1)	1,923	20.1%
Neutral (2)	6,178	64.7%

Table 1: Training-set class distribution.

Tweet lengths range from 2 to 48 tokens (median: 18), and all three classes share similar length distributions (Figure 1), so length itself is not discriminative. This also justifies our `max_length=128` setting for the Transformer tokenizers — no tweet approaches that limit.

Financial tweets carry distinctive surface markers: \$TICKER cashtags appear in $\sim 60\%$ of tweets, @mentions in $\sim 15\%$ and URLs in $\sim 20\%$. Cashtags carry real contextual signal, but overfitting to specific tickers would not generalise, so we replace them with a `<TICKER>` placeholder.



Analysis of the most frequent tokens per class (Figure 2) reveals clear thematic separation: Bearish tweets feature words such as “cut”, “downgrade” and “sell”; Bullish tweets contain “buy”, “upgrade” and “breakout”; Neutral tweets use informational vocabulary (“earnings”, “report”, “market”) that overlaps with both sentiment classes, making the Neutral boundaries the primary source of classification error.

Data-quality checks found fewer than 1% exact duplicates (retained, to avoid biasing the class proportions) and no label leakage: “bearish” and “bullish” appear naturally in financial discourse and represent legitimate patterns rather than annotation shortcuts.

3. Data Preprocessing

Preprocessing was implemented as a set of composable functions in `src/preprocessing.py`, which we combine into three named pipelines and benchmark systematically.

3.1 Preprocessing Functions

Each cleaning step is a standalone function so that pipelines can be assembled and ablated:

- ## 1. **HTML unescape** — decode `&`, `<`, ...

2. **URL removal** — strip `http://` and `www.` links.
3. **Mention removal** — strip `@username` handles.
4. **Hashtag handling** — strip the leading `#` but keep the word.
5. **Ticker normalisation** — replace cashtags with `<TICKER>`.
6. **Number normalisation** — percentages $\rightarrow$ `<PCT>`, numbers $\rightarrow$ `<NUM>`.
7. **Emoji demojisation** — convert emojis to text descriptions, preserving sentiment signal.
8. **Unicode NFKC** — consistent encoding of look-alike characters.
9. **Lowercasing** — standard case folding.
10. **Punctuation collapse** — `!!!` $\rightarrow$ `!`; whitespace normalisation.
11. **Finance-aware stopword removal** — we keep negations (`no`, `not`, `nor`) and directional words (`up`, `down`, `over`, `under`); these are critical for sentiment polarity.
12. **Lemmatisation / stemming** — evaluated as optional token-level operations.

3.2 Named Pipelines

Pipeline	Steps	Use case
<code>pp_minimal</code>	1–10	BoW, TF-IDF, Word2Vec
<code>pp_aggressive</code>	1–12	Tested and dropped
<code>pp_transformer</code>	1–7, 10	Transformer inputs

Table 2: Preprocessing pipelines and their target use cases.

3.3 Pipeline Comparison

We benchmarked the pipelines under a single fixed classifier (Logistic Regression on TF-IDF features) to isolate the preprocessing effect (Table 3).

Pipeline	Macro-F1
<code>pp_minimal</code>	0.736
<code>pp_aggressive</code> (lemma)	0.718
<code>pp_aggressive</code> (stem)	0.709

Table 3: Preprocessing-pipeline comparison (LogReg + TF-IDF baseline).

`pp_minimal` consistently outperforms `pp_aggressive`: the aggressive pipeline destroys discriminative signal. Stopword removal strips negations such as “not” that flip sentiment, lemmatisation collapses temporal distinctions (“falling” $\rightarrow$ “fall”) that carry urgency, and number normalisation discards specific values that provide context.

4. Feature Engineering

We implemented and compared eight feature-extraction methods across three families.

4.1 Bag-of-Words Features

CountVectorizer (BoW): unigram counts with a 10 000-feature vocabulary cap. **TF-IDF**: bigrams (`ngram_range=(1,2)`), sublinear TF scaling and 20 000 features with `min_df=2` and `max_df=0.95`. TF-IDF outperformed BoW by 1–3 percentage points across all classifiers, confirming that local word order carries sentiment signal (Figure 3).

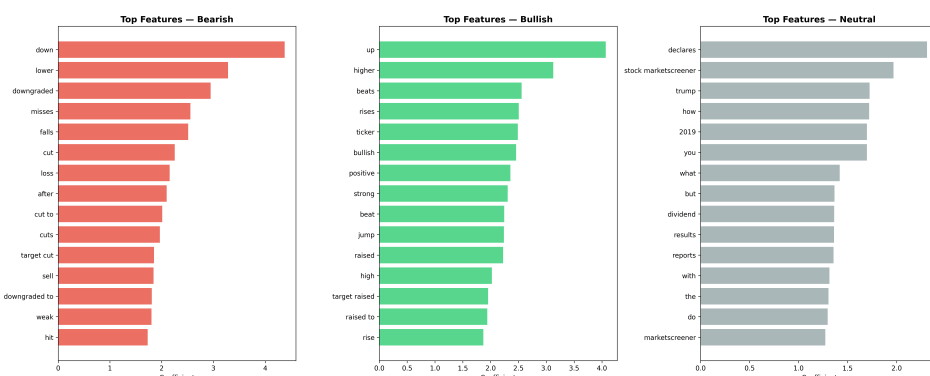


Figure 3: Top TF-IDF features per class (Logistic Regression coefficients).

4.2 Word Embeddings

Custom Word2Vec: skip-gram, 100-d, trained on the financial tweet corpus, mean-pooled. **GloVe Twitter 100-d**: pretrained on 2 billion tweets. **TF-IDF-weighted pooling**: weights each token embedding by its TF-IDF score.

All Word2Vec variants underperform BoW/TF-IDF (macro-F1: 0.59–0.66 vs. 0.72–0.74). The primary reason is that mean pooling discards word order, which is critical for sentiment: averaged GloVe-Twitter vectors for “stock is going up today” and “stock is going down today” have a cosine similarity of **0.996** — two sentences of opposite financial polarity, indistinguishable to any classifier downstream of the pooled representation.

4.3 Transformer Encoders

We used three pretrained Transformer encoders as frozen feature extractors: tokenise, pass through the encoder, mean-pool the last hidden states (768-d) with attention masking and feed the result to a downstream scikit-learn classifier. Embeddings are cached as `.npy` files, which keeps the workflow CPU-feasible.

Key finding. Domain alignment matters more than model size. DistilBERT (general-purpose, 66M params) underperforms TF-IDF; Twitter-RoBERTa (tweet-

Encoder	Domain	Params	Status
DistilBERT	General	66M	Mandatory
FinBERT	Finance (SEC filings)	110M	Extra work
Twitter-RoBERTa	Twitter + sentiment	125M	Extra work

Table 4: Transformer encoders evaluated as frozen feature extractors.

domain, sentiment-tuned, 125M params) achieves the best result overall; FinBERT sits between the two — it understands financial language but was trained on formal SEC filings rather than informal tweets.

Tokenisation. The three encoders use WordPiece (DistilBERT, FinBERT) or byte-pair (Twitter-RoBERTa) vocabularies, so most finance vocabulary is split: `bullish` $\rightarrow$ `bull|ish`, `downgrade` $\rightarrow$ `down|grade`, `rebalancing` $\rightarrow$ `re|bala|nc|ing` (BERT-family) or `re|bal|ancing` (RoBERTa). The `<TICKER>` placeholder is fragmented across all three tokenizers, so the encoder loses the holistic “ticker” token but still sees a consistent rare-token pattern across cashtags — which is the property the placeholder was meant to preserve.

Pooling. The canonical sentence representation in BERT-family models is the `[CLS]` token. We used mean pooling and verified directly that this is not a performance compromise (Table 5): for the sentiment-tuned encoders the two pooling choices are within noise, and mean pooling slightly helps DistilBERT, whose `[CLS]` representation has not been fine-tuned for any sentiment task.

Encoder	Mean pool	CLS pool	Δ
Twitter-RoBERTa	.794 $\pm$.006	.792 $\pm$.010	-.002
FinBERT	.779 $\pm$.003	.776 $\pm$.007	-.003
DistilBERT	.674 $\pm$.012	.691 $\pm$.010	+.017

Table 5: Mean vs. CLS pooling on the last hidden states (XGBoost, 5-fold CV).

5. Classification Models

We evaluated four classifiers (Table 6). All use class weighting to handle imbalance, 5-fold stratified cross-validation and `random_state=42` for reproducibility.

Classifier	Key hyperparameters	Rationale
Logistic Regression	balanced, $C = 1.0$, multinomial	Interpretable linear baseline
LinearSVC	balanced, $C = 1.0$	Maximum-margin, sparse data
XGBoost	300 trees, depth 6, softmax	Non-linear decision boundaries
Random Forest	300 trees, balanced	Ensemble baseline

Table 6: Classifiers and their key hyperparameters.

6. Results and Analysis

6.1 Full Results Matrix

Table 7 reports macro-F1 across all 32 combinations.

Features	LogReg	SVM	XGBoost	RandForest
BoW	.720±.011	.708±.010	.691±.013	.664±.012
TF-IDF	.736±.006	.740±.013	.670±.011	.654±.014
W2V-mean	.600±.013	.603±.012	.650±.006	.595±.011
W2V-tfidf	.595±.013	.610±.015	.658±.016	.611±.011
GloVe-mean	.599±.017	.617±.014	.636±.009	.494±.004
DistilBERT	.702±.009	.705±.008	.674±.012	.517±.007
FinBERT*	.736±.007	.751±.009	.779±.003	.734±.002
Twitter-RoBERTa*	.755±.011	.764±.008	.794±.006	.742±.008

Table 7: Macro-F1 (mean ± std) over 5-fold CV. Bold: best per row; underline: overall best.
* extra work.

6.2 Key Findings

Feature hierarchy (most to least effective):

1. **Twitter-RoBERTa** (0.794) — pretrained on 58M tweets with sentiment labels; complete domain alignment.
2. **FinBERT** (0.779) — finance-aware, but pretrained on formal text rather than tweets.
3. **TF-IDF** (0.740) — competitive: bigrams recover some local word order.
4. **BoW** (0.720) — solid unigram baseline.
5. **DistilBERT** (0.705) — general-purpose encoder underperforms TF-IDF, illustrating that domain alignment matters more than model sophistication.
6. **Word2Vec variants** (0.658) — mean pooling discards word order.

Classifier patterns. XGBoost excels with dense Transformer embeddings (non-linear boundaries in 768-d space), while linear models (LogReg, SVM) excel with sparse features (BoW/TF-IDF), where regularisation handles high dimensionality well. Random Forest consistently underperforms both.

Standard deviations remain below 0.017, and the best configuration carries one of the lowest (± 0.006) — the ranking is stable across folds.

6.3 Per-Class Analysis

Neutral is consistently the easiest class ($F1 > 0.87$) thanks to a large support and distinctive factual vocabulary. Bearish and Bullish are harder ($F1$: 0.65–0.75) because they share domain vocabulary and differ primarily in polarity. Bearish is slightly harder

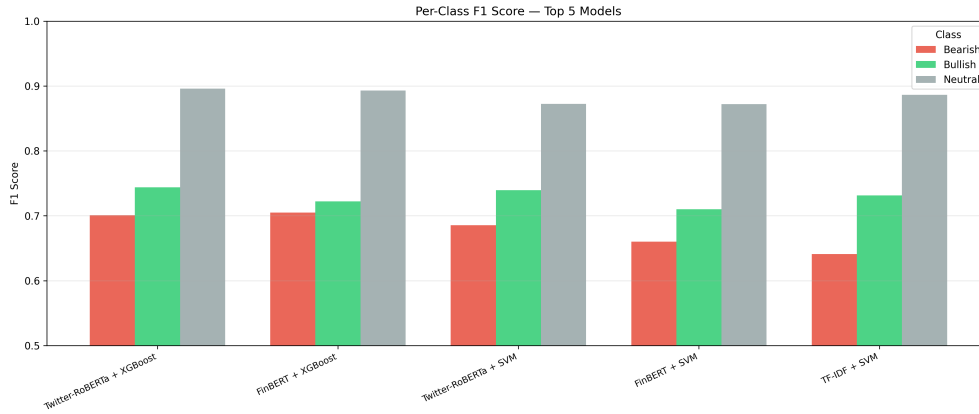


Figure 4: Per-class F1 for the top-5 configurations.

still, since bearish tweets sometimes use hedging language (“might drop”) that resembles neutral commentary.

6.4 Embedding Geometry

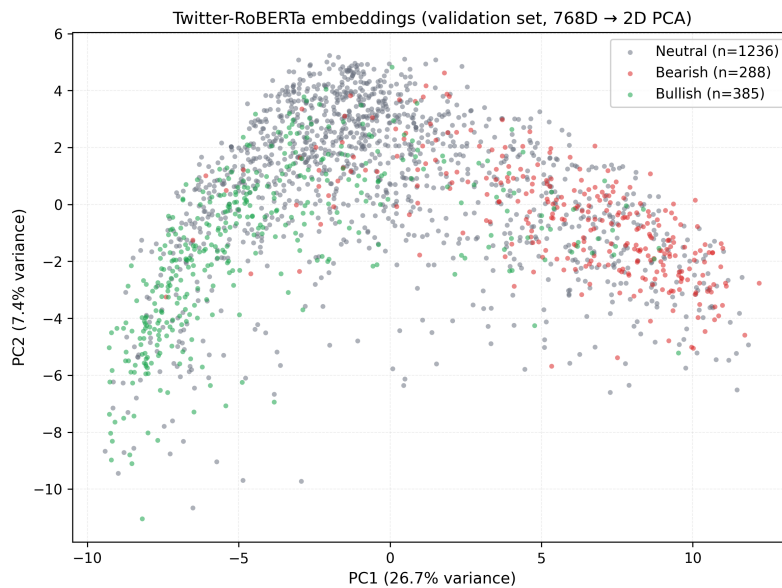


Figure 5: PCA projection of Twitter-RoBERTa embeddings on the validation set (768-d $\rightarrow$ 2-d).

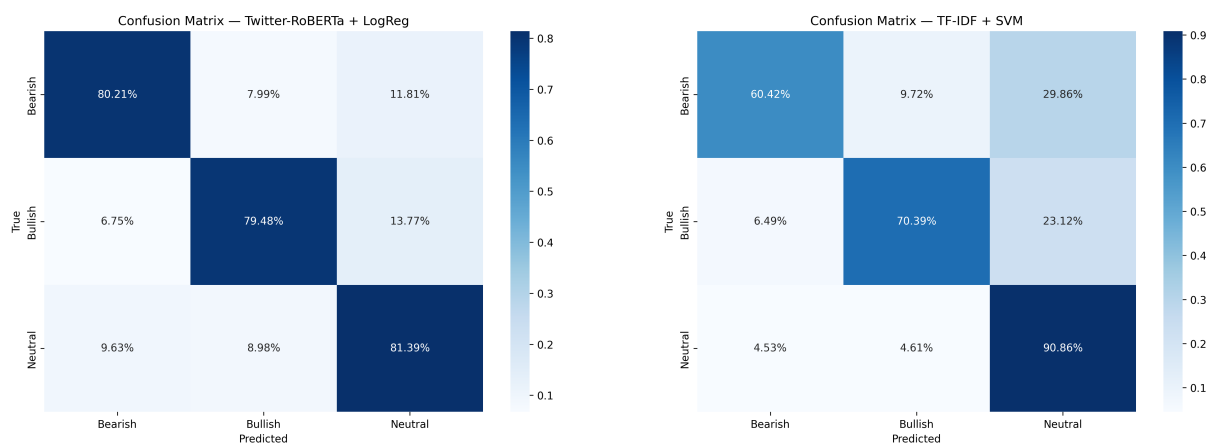
To check whether the best encoder genuinely separates the three classes in feature space, we projected the 768-d Twitter-RoBERTa embeddings of the validation set to 2-d via PCA (Figure 5). PC1 alone accounts for 26.7% of variance and roughly tracks the Bearish $\leftrightarrow$ Bullish axis; Neutral occupies the centre of the cloud, mirroring the dominant error mode in the confusion matrices (Neutral $\leftrightarrow$ sentiment confusion). The clusters overlap but remain visibly distinct — this is the geometry that XGBoost exploits to reach 0.79 macro-F1.

6.5 Error Analysis

The misclassified tweets fall into three recurring patterns, visualised in the confusion matrices of Figure 6:

1. **Neutral** ↔ **Bearish**: tweets reporting negative news factually (e.g. “Stock down 5% after earnings miss”) are genuinely ambiguous between factual reporting and negative sentiment.
2. **Neutral** ↔ **Bullish**: factual reports of positive events blur the line between informational and positive sentiment.
3. **Bearish** ↔ **Bullish** (rarest): primarily sarcasm or contrarian positions.

The dominant challenge is the fundamental ambiguity between factual reporting and sentiment expression in financial discourse.



(a) Twitter-RoBERTa + XGBoost (best overall).

(b) TF-IDF + SVM (best traditional).

Figure 6: Confusion matrices for the best Transformer and best traditional model.

6.6 Learning Curve and Statistical Significance

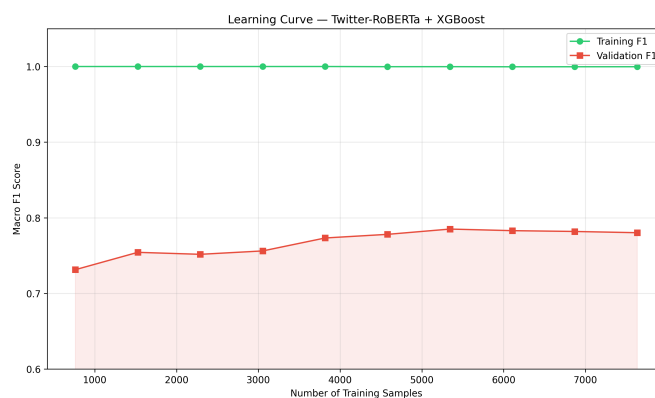


Figure 7: Learning curve for Twitter-RoBERTa + XGBoost.

Validation F1 plateaus after roughly 40% of the training data (Figure 7), suggesting the model is near its ceiling for this feature representation. The persistent train–validation gap indicates mild overfitting that more data alone will not resolve — architectural improvements (full fine-tuning) would be needed.

Paired t -tests on the 5-fold CV scores confirm that the gaps between the top three configurations are statistically significant ($p < 0.05$): Twitter-RoBERTa+XGBoost vs. FinBERT+XGBoost; Twitter-RoBERTa+XGBoost vs. TF-IDF+SVM; and FinBERT+XGBoost vs. TF-IDF+SVM. The observed ranking is not an artefact of random fold variation.

7. Extra Work

7.1 Domain-Specific Transformer Encoders

[EXTRA WORK] We evaluated two pretrained encoders beyond the mandatory DistilBERT. **FinBERT** (ProsusAI/finbert), trained on financial communications, reaches **0.779** macro-F1 (a 7.4-pp improvement over DistilBERT). **Twitter-RoBERTa** (cardiffnlp/twitter-roberta-base-sentiment-latest), trained on 58M tweets with sentiment labels, reaches **0.794** macro-F1 — the best overall.

7.2 Zero-Shot Pretrained Classification

[EXTRA WORK] To isolate the contribution of the trained downstream classifier from that of the pretrained encoder, we applied Twitter-RoBERTa’s built-in 3-class sentiment head (negative/neutral/positive) directly to the validation set, with no training of our own. Outputs were mapped to our label space: negative → Bearish, positive → Bullish, neutral → Neutral.

The zero-shot baseline reaches **0.587** macro-F1 (5-fold aggregation: 0.587 ± 0.012), substantially below the 0.794 obtained when the same encoder is paired with a trained XGBoost classifier (Table 8). The gap concentrates on the minority classes: Bearish/Bullish F1 fall from 0.65–0.75 (trained) to 0.48–0.49 (zero-shot), while Neutral degrades less. This reflects a label-space mismatch: a tweet using “downgrade”, “cut” or “miss” is bearish in finance but not unambiguously negative in everyday speech. Supervised adaptation on ~ 9.5 k labeled examples is what recovers the missing +0.21 macro-F1; pretraining domain alignment is necessary but not sufficient.

7.3 Agentic Workflow

[EXTRA WORK] A LangChain-based agent (`src/agent.py`) orchestrates five tools: `classify_tweet`, `route_model`, `ensemble_classify`, `compare_models` and `explain_prediction`. The agent inspects tweet content to choose a model, rec-

Approach (Twitter-RoBERTa encoder)	Macro-F1
Zero-shot (pretrained 3-class head, no training)	0.587
Frozen embeddings + XGBoost (ours)	0.794
Δ from supervised adaptation	+0.207

Table 8: Same encoder, two heads: zero-shot vs. supervised adaptation.

onciles disagreements via majority vote and produces interpretable explanations from linear-model coefficients. Three illustrative transcripts (full logs in `outputs/agent_transcripts.md`):

Routing. Input “\$AAPL all-time high, breakout above resistance.” — the agent detects a \$TICKER, routes to FinBERT and returns Bullish (89.1%).

Ensemble. Input “Markets mixed; Fed signals rate changes amid inflation concerns.” — TF-IDF/LogReg, FinBERT and Twitter-RoBERTa vote Neutral while DistilBERT votes Bearish; the agent returns Neutral with 75% agreement, illustrating how a general-purpose encoder can over-weight non-financial vocabulary.

Attribution. Input “Sell everything, this stock is going to crash hard.” — LogReg coefficients identify “sell” (+2.85) and “crash” (+2.63) as the strongest Bearish signals, both carrying negative Bullish weights (−1.92, −2.10).

7.4 Decoder Model (Attempted)

[EXTRA WORK] Few-shot classification with Flan-T5-small was implemented but CPU inference proved infeasible ($\sim 2\text{--}5$ s per tweet, > 3 h for the test set); the result is documented as a CPU-only limitation rather than a delivered baseline.

8. Final Pipeline and Predictions

The production pipeline (`tm_final_42.ipynb`) retrains the best model on the full training set (9543 tweets) and generates predictions for all 2388 test tweets. The steps are: (i) preprocess all data with `pp_transformer`; (ii) extract Twitter-RoBERTa embeddings (cached as `.npy`); (iii) train XGBoost on the full training embeddings; (iv) predict on the test embeddings $\rightarrow$ `pred_42.csv`.

Class	Pred. count	Pred. %	Train %
Bearish	309	12.9%	15.1%
Bullish	392	16.4%	20.1%
Neutral	1 687	70.6%	64.7%

Table 9: Test-set prediction distribution vs. training proportions.

The prediction distribution closely matches the training proportions, indicating that the model is neither over- nor under-predicting any class.

9. Conclusion

The systematic evaluation of 32 feature $\times$ classifier combinations identifies **Twitter-RoBERTa + XGBoost** as the best pipeline (**0.794** macro-F1). Four findings stand out:

1. **Domain alignment outweighs model size.** DistilBERT (66M params) underperforms TF-IDF bigrams; Twitter-RoBERTa (125M params) exceeds both. Alignment alone is not enough either — the same encoder, applied zero-shot (Section 7.2), reaches only 0.587 macro-F1; the trained head adds +0.21.
2. **Feature extraction matters more than classifier choice.** The gap between feature families (0.60 $\rightarrow$ 0.79) far exceeds the within-family gap (0.02–0.05).
3. **Conservative preprocessing wins.** Aggressive normalisation removes sentiment signal; keeping negations and directional words intact is essential.
4. **Class-imbalance handling is essential.** Without balanced class weights, classifiers default to predicting Neutral.

Limitations and future work. The CPU-only constraint prevented full Transformer fine-tuning and made decoder inference infeasible; no hyperparameter optimisation was performed; results come from a single dataset. GPU fine-tuning, cross-family ensembles and temporal analysis of financial sentiment are the natural next steps.